

Actor and Action Modular Network for Text-Based Video Segmentation

Jianhua Yang^{ID}, Yan Huang^{ID}, Kai Niu^{ID}, Linjiang Huang^{ID}, Zhanyu Ma^{ID}, *Senior Member, IEEE*,
and Liang Wang, *Fellow, IEEE*

Abstract—Text-based video segmentation aims to segment an actor in video sequences by specifying the actor and its performing action with a textual query. Previous methods fail to explicitly align the video content with the textual query in a fine-grained manner according to the actor and its action, due to the problem of *semantic asymmetry*. The *semantic asymmetry* implies that two modalities contain different amounts of semantic information during the multi-modal fusion process. To alleviate this problem, we propose a novel actor and action modular network that individually localizes the actor and its action in two separate modules. Specifically, we first learn the actor-/action-related content from the video and textual query, and then match them in a symmetrical manner to localize the target tube. The target tube contains the desired actor and action which is then fed into a fully convolutional network to predict segmentation masks of the actor. Our method also establishes the association of objects cross multiple frames with the proposed temporal proposal aggregation mechanism. This enables our method to segment the video effectively and keep the temporal consistency of predictions. The whole model is allowed for joint learning of the actor-action matching and segmentation, as well as achieves the state-of-the-art performance for both single-frame segmentation and full video segmentation on A2D Sentences and J-HMDB Sentences datasets.

Index Terms—Video object segmentation, language attention mechanism, modular network, multi-modal learning.

Manuscript received 15 September 2021; revised 12 April 2022; accepted 2 June 2022. Date of publication 28 June 2022; date of current version 1 July 2022. This work was supported in part by the National Key Research and Development Program of China under Grant 2018AAA0100400; in part by the Key Research Program of Frontier Sciences Chinese Academy of Sciences (CAS) under Grant ZDBS-LY-JSC032; in part by the National Natural Science Foundation of China under Grant 61721004, Grant U1803261, Grant 61976132, Grant 61922015, and Grant U19B2036; in part by the Beijing Nova Program under Grant Z201100006820079; in part by the Chinese Academy of Sciences-Artificial Intelligence Research (CAS-AIR); in part by the Beijing Natural Science Foundation Project under Grant Z200002; and in part by the Open Projects Program of National Laboratory of Pattern Recognition under Grant 202100028. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Xiangqian Wu. (*Corresponding author Liang Wang.*)

Jianhua Yang and Zhanyu Ma are with the Pattern Recognition and Intelligent Systems Laboratory, School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: youngjianhua@bupt.edu.cn; mazhanyu@bupt.edu.cn).

Yan Huang and Liang Wang are with the Center for Research on Intelligent Perception and Computing (CRIPAC), National Laboratory of Pattern Recognition (NLPR), Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing 100190, China, and also with the School of Artificial Intelligence, University of Chinese Academy of Sciences (UCAS), Beijing 100049, China (e-mail: yhuang@nlpr.ia.ac.cn; wangliang@nlpr.ia.ac.cn).

Kai Niu is with the School of Computer Science, Northwestern Polytechnical University, Xi'an 710072, China (e-mail: kai.niu@nwpu.edu.cn).

Linjiang Huang is with the Multimedia Laboratory, The Chinese University of Hong Kong, Hong Kong 999077 (e-mail: ljhuang524@gmail.com).

Digital Object Identifier 10.1109/TIP.2022.3185487

I. INTRODUCTION

WITH the explosion of video data on the Internet, understanding video content becomes increasingly important and has attracted significant research interest in recent years. Action recognition is one of the key tasks in the field of video analysis, which mainly focuses on human-centric action classification [1], [2] or localization [3], [4] in videos. Since traditional action recognition lacks fine-grained understanding of video content, recently there is a growing interest in simultaneously understanding actors and actions in videos. Xu *et al.* [5] first proposed a new actor-action segmentation challenge on a large-scale video dataset, *i.e.*, actor-action dataset (A2D). Actor-action semantic segmentation requires to spatio-temporally localize and recognize seven classes of actors (*e.g.*, “adult” and “bird”) and eight classes of actions (*e.g.*, “climb” and “fly”) at pixel-level in a video. Various approaches under supervision [6]–[11] or weak supervision [12], [13] have been proposed to tackle this problem and achieved significant advance. However, the actors and actions of interest are much more diverse in the real world, thus the small number of pre-defined actor and action classes largely limits applications of the aforementioned methods in some cases, such as automatic video editing, intelligent vision search, and human-robot interaction.

Compared with word-level actor and action classes, textual queries are more flexible so that they can be used to specify various actors and actions of interest. While most related works focus on either spatial object localization from textual queries in static images [14]–[16] or temporal action localization from textual queries in videos [17], [18]. There are few works that consider the actor and the corresponding action simultaneously in a video when the actor and action are specified by a given textual query. To bridge this gap, the new task of text-based video segmentation was originally introduced by Gavriluk *et al.* [19], whose goal was to attain pixel-level segmentation of the actor in video content by specifying the actor and its performing action with a textual query. Some illustrative examples of the new task are shown in Fig. 1, where four textual queries are given to specify four different actors and their actions, respectively. The model is required to predict segmentation masks of actors frame-by-frame conditioned by the textual queries. The variety of textual queries provides a flexible way to select the actors of interest according to actors’ appearance information and motion information. Appearance information contains visual



Fig. 1. Illustration of text-based video segmentation. The goal of the task is to attain segmentation masks of the actor in video sequences by specifying the actor and its action with a textual query. Note that the mask color corresponds to the textual query color (Better viewed in color).

cues about categories, colors, and shapes of actors, while motion information represents actions that are performed by actors in the video. Considering either video or textual query understanding has been extensively studied in its related fields, the major challenge of this task is to establish a suitable video-query alignment by associating with actors and actions. Despite much progress has been achieved in this task, existing methods [19]–[22] suffer from the following two aspects.

First, the fine-grained alignment between video and query features is not explored well. Existing methods adopt a bottom-up paradigm to address the text-based video segmentation, they fail to capture entity-level (*i.e.*, actor and action) information for explicitly modeling the fine-grained alignment between two modalities. More concretely, these methods first integrate video and query features with various fusion strategies, and then they implicitly align each convolved local volume (*i.e.*, Spatio-temporal feature unit) of the 3D feature map produced by a 3D convolutional neural network (CNN) with a holistic representation of the textual query. Finally, they measure the binary similarity map from the decoder network. However, for the query representation, the arbitrary input query could include various words referring to actors, actions, attributes, locations, objects, *etc.*, which contains much actor-/action-related information. Besides, the variation of linguistic structures of queries makes these methods difficult to precisely capture the semantic information related to the actor and its action. For the video features, the convolved local volume usually provides either redundant or incomplete visual content for a single actor depending on relative sizes of the actor and convolutional filters. In addition, since each local volume could not include sufficient information to recognize an action, modeling spatio-temporal relationships among local volumes [23], [24] is helpful to recognize the action. Therefore, the local volume of the video features and the holistic representation of the textual query contain different amounts of semantic information, namely, *semantic asymmetry* between the local volume and the query representation. The *semantic asymmetry* fails to align the textual query with referred actor and its action in video content, and eventually degrades segmentation results.

Second, existing methods are inefficient for full video segmentation and suffer from the discrepancy of predictions in consecutive frames, as they do not consider object association between adjacent frames. As A2D dataset [5]

is sparsely annotated 3 to 5 frames for a video, existing methods [19]–[21] directly aggregated the spatio-temporal features into the annotated frames to train and evaluate their models. Consequently, there are two limitations for these methods: 1) They are time-consuming to segment all frames of the video by densely sampling lots of video clips, because their models only predict one-frame segmentation results using 2D decoder networks with multi-frame video clips as inputs. 2) The temporal consistency of predictions cross multiple frames is not established well as the outputs of the models are treated independently. The scene changes and motion blur would lead to the discrepancy of predictions in consecutive frames. To alleviate these issues, McIntosh *et al.* [22] replaced 2D deconvolutional layers with 3D deconvolutional layers in the decoder network, and used annotations of bounding boxes to train and evaluate their model. However, the block-like outputs from the model are not precise enough for pixel-level actor segmentation in videos.

To overcome the limitation of *semantic asymmetry*, an intuitive idea is to first find actor-/action-related video content and words, rather than equally split local volumes of video features and directly use the holistic representation from the textual query. This can be easily achieved by leveraging the recent advances of object detection [25]–[27] and language attention mechanism [28]–[30]. Then the cross-modal matching between extracted instance-level regions and actor-related words is straightforward, as it is similar to the referring expression methods [29], [30]. If the region contains the actor referred to by the query, namely, *semantic symmetry* between the region and the actor-related words, it is desired region. However, the referring expression methods have two drawbacks for text-based video segmentation. On the one hand, the referring expression methods do not refer to the dynamic information or actions of the actor, the action-related matching is still a challenge. The action-related matching additionally requires understanding the dynamic motion rather than only static actors. It is more complicated as the video content also includes other disturbing relationships, *e.g.*, affiliation and position, which can easily lead to confusion. Moreover, the actor-related matching and action-related matching are closely related, *i.e.*, it is the actor that actually performs the action. But how to make them collaborate with each other to improve the final performance is rarely investigated. On the other hand, the referring expression methods fail to establish data association for objects cross multiple frames. These methods process the video frame-by-frame independently without considering the valuable temporal information cross frames, which could result in the discrepancy of predictions due to the scene or appearance variations in the video.

In this paper, we propose a novel model which named Actor and Action Modular Network (AAMN) for text-based video segmentation. Specifically, to effectively utilize the limited annotations and establish the association of objects in consecutive frames, we first generate a set of actor-/action-related tubes by introducing a temporal proposal aggregation mechanism. This allows us to obtain appearance and motion representations for each tube of the clip. In order to realize semantically symmetrical matching, we implement the language attention mechanism on the textual query to adaptively focus on actor-

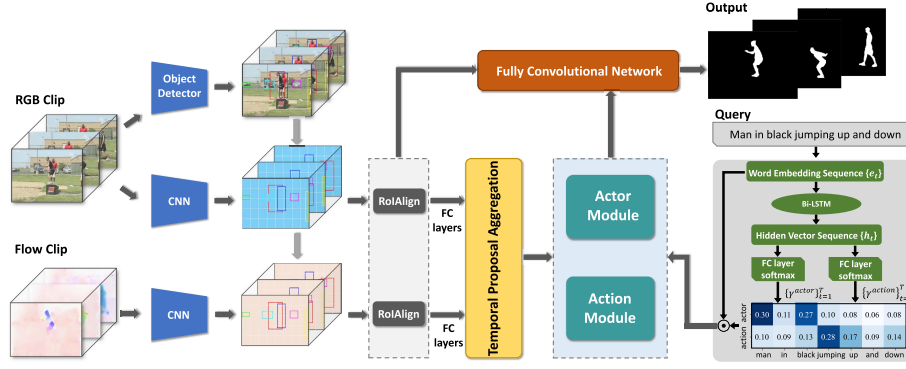


Fig. 2. An overview of our proposed actor and action modular network (AAMN) for text-based video segmentation (Better viewed in color). The AAMN takes a textual query, RGB and Flow clips as well as corresponding proposals generated from an external detector as inputs. The temporal proposal aggregation mechanism will generate a set of actor-/action-related tubes by computing linking scores. The language attention model will adaptively learn the actor-/action-related representation from the given textual query. Two modules are proposed to localize the target tube that involves the actor and action referred to by the textual query. The fully convolutional network (FCN) is used to predict the masks of the actor within the target tube.

/action-related words, and then construct two modules in AAMN. One module is called actor module, which is designed to identify the actor based on actor's appearance information. It associates actor-related tubes with the corresponding words to produce actor matching scores. Another module is called action module, which is designed to identify the actor based on actions performed by the actor in the video. This module utilizes optical flow features to model the dynamic motion information of the actor. To further keep our model from being disturbed by ambiguous motion information or background, the action module contains a contextual long short-term memory network (LSTM) to model the potential relationships among pairwise tubes. The action module integrates motion features and contextual features as well as action-related words to predict the action-matching scores. Finally, the feature maps of the target tube are fed into a small fully convolutional network (FCN) [31] to predict masks of the input clip. Our method can not only achieve single-frame video segmentation like previous methods in [19]–[21], but also full video segmentation like the method in [22]. To demonstrate the effectiveness of the proposed AAMN, we conduct experiments on two benchmark datasets, and experimental results show that our method outperforms the state-of-the-art methods.

The contributions of this paper are highlighted as follows:

- i) We study on a practically important problem, *i.e.*, text-based video segmentation, and accordingly propose a novel Actor and Action Modular Network (AAMN) to deal with the intrinsic *semantic asymmetric* problem.
- ii) We propose a temporal proposal aggregation mechanism to establish objects association cross adjacent frames. This enables AAMN to achieve multi-frame segmentation at a time with limited annotations, and keep the temporal consistency of predictions in consecutive frames.
- iii) We especially focus on the action-related matching between the video content and the textual query, by constructing an action module to jointly model the dynamic motion and visual context.
- iv) We quantitatively and qualitatively validate the effectiveness of our method and achieve the state-of-the-art results on A2D Sentences and J-HMDB Sentences datasets.

II. RELATED WORK

A. Actor-Action Semantic Segmentation in Videos

There are many emerging works on video object segmentation that aims to segment out a particular object in an entire video [32]. Actor-action semantic segmentation in videos differs from them by assigning an action label to the target actor. Such a detailed actor-action understanding task has attracted growing attention in recent years. Xu *et al.* made the first effort on actor-action semantic segmentation problem in [5], where they collected a large-scale video dataset, *i.e.*, A2D, to jointly consider various types of actors performing various actions. Early works [5], [6] solved this problem based on Conditional Random Fields (CRF) with supervoxels features. With the success of deep learning in the field of computer vision, deep learning-based approaches have been widely studied for actor-action semantic segmentation. Qiu *et al.* [7] exploited 2D and 3D FCN to model spatio-temporal dependency from a video for semantic segmentation. Dang *et al.* [9] relied on region masks to enhance consistent labeling of action for actor-action semantic segmentation. Kalogeiton *et al.* [8] proposed a multitask architecture for joint actor-action detection and then performed actor-action segmentation with SharpMask [33]. Ji *et al.* [10] leveraged multiple input modalities and contextual information from videos to realize pixel-wise actor-action segmentation and detection in a unified network with joint multitask learning. Rana *et al.* [11] proposed an effective method to perform pixel-level actor-action detection in a single-shot. Moreover, the weakly supervised actor-action semantic segmentation, which only has access to video-level actor and action tags, has also been studied in recent works [12], [13].

Although promising results have been achieved by the aforementioned approaches, the pre-defined actor and action categories limit their applications in human-computer interaction scenarios. Our work focuses on solving a more challenging task, namely, text-based video segmentation, which aims to segment a particular actor specified by a textual query referring to the actor and its performing action in the video.

B. Referring Expression Comprehension and Segmentation

The referring expression comprehension (REC) aims to localize the object with a bounding box in the image that

corresponds to a given textual query [34]. To address this problem, most of proposed approaches focus on mining contextual information from the language and image [14], [35], modeling the relationship between different objects [30], [36], and analyzing the linguistic structures [29]. To precisely describe the shape of the referred object, the referring expression segmentation (RES) goes a step further to predict the segmentation mask instead of the bounding box. Hu *et al.* first proposed the baseline by up-sampling the concatenation of visual and linguistic features using a deconvolutional layer. Margffoy-Tuay *et al.* [37] integrated the visual and linguistic features by generating the dynamic filter for each word of the textual query. Li *et al.* [38] explored to fuse multi-level visual features to recurrently refine the local details of segmentation masks. To enable better multi-modal interactions, more recent methods [39]–[44] focus on establish complex cross-modal relations between visual and linguistic features. For example, Ye *et al.* [41] integrated every word with multi-level visual features and introduce self-attention to adaptively focus on informative words of query and regions of the image. Hu *et al.* [42] proposed a bi-directional attention module to establish cross-modal correlations, Liu *et al.* [43] leveraged attention mechanism to capture different concepts for enhancing the multi-modal fusion process.

Text-based video segmentation is closely related to the task of RES, some works [41], [43] also explored to address text-based video segmentation with RES methods. However, there are still significant differences between these two tasks. *First*, for the RES, the target object is mainly referred to by its appearance or location in a given static image. While for text-based video segmentation, the target object is referred to by not only its attributes (*e.g.*, colors, shapes), but also its performing actions in the video. This indicates that the text-based video segmentation is more complicated than RES. *Second*, RES methods segment the video frame-by-frame without considering the data association between adjacent frames, which usually leads to the discrepancy of predicted objects. It is essential to establish the association of objects cross multiple frames for text-based video segmentation. *Third*, text-based video segmentation adopts more metrics to evaluate the segmentation performance except the overall intersection over union that is often used in RES.

C. Text-Based Video Segmentation

The task of text-based video segmentation was originally introduced by Gavrilyuk *et al.* [19], whose goal was to segment the actor specified by a textual query referring to the actor and its performing action in the video. Existing approaches address this problem by following a bottom-up procedure, which extracts the video and query features separately and then predicts the segmentation mask from the merged heterogeneous features. Specifically, Gavrilyuk *et al.* [19] proposed to correlate the video contents and sentences using dynamic convolutions. Wang *et al.* [20] introduced vision guided language attention and language guided vision attention mechanisms to obtain a robust language representation and aggregate global visual context for performance gains. Wang *et al.* [21]

constructed a context modulated dynamic convolutional network to incorporate contextual information into the correlation learning of heterogeneous modalities. McIntosh *et al.* [22] further encoded the visual and language features as capsules and integrated the visual and language information via a routing algorithm. Unlike aforementioned works, we propose to solve the text-based video segmentation from another perspective, namely, a top-down procedure. We first localize the referred actor and action with bounding boxes sequence in a symmetric matching way, then predict the masks of the actor along the bounding boxes sequence. Our method can effectively localize the query described actor and action, and reduce disturbances of irrelevant objects and the background in videos.

III. METHODOLOGY

In this section, we elaborate on the proposed method for text-based video segmentation, the overall framework of our method is illustrated in Fig. 2.

Task Definition: Given a video sequence $\mathcal{I} = \{I_n \in \mathbb{R}^{W \times H \times 3}\}_{n=1}^N$ with N frames, and a corresponding textual query $\mathcal{W} = \{w_t\}_{t=1}^T$ with T words. The textual query usually refers to the actor and its performing action in the video. The text-based video segmentation aims to produce N -frame binary segmentation masks $\mathcal{M} = \{M_n\}_{n=1}^N$, $M_n \in \mathbb{R}^{H \times W}$ for the referred actor in the video sequence.

Overview: Except appearance information, which contains visual cues about categories, colors and shapes of the actors, dynamic motion information is another important visual cue to identify the actors in the video. This motivated us to localize the actor cross frames by aligning appearance information and motion information between the video content and the textual query. To explicitly model such fine-grained alignment and achieve semantically symmetric matching between two modalities, we propose an actor and action modular network (AAMN) for text-based video segmentation.

Formally, our AAMN first generates a set of proposals per frame in a video clip via an external object detector. Then AAMN feeds RGB and Flow clips as well as corresponding proposals into two parallel CNN architectures to learn the appearance and motion representations for each proposal. To link the proposals cross frames and generate a set of actor-/action-related tubes for input clips, a temporal proposal aggregation mechanism is introduced after fully connected (FC) layers (Sec. III-A). Meanwhile, in order to obtain related representations about the actor and action from the textual query, AAMN decomposes the query into two separate components relying on a language attention mechanism (Sec. III-B). Afterward, we construct two modules, *i.e.*, actor module and action module, to identify the actor based actor's appearance information and motion information, respectively. These two modules take actor-/action-related tubes and language representations as inputs and localize the target tube according to the actor and action matching scores (Sec. III-C). Finally, a tiny FCN is followed by two modules to predict the segmentation masks of the actor within the target tube (Sec. III-D). We train the whole network with a multi-task learning strategy by combining the actor-action matching and the actor segmentation (Sec. III-E).

A. Actor-/Action-Related Tubes Generation and Representation

The AAMN takes RGB and Flow clips as well as pre-generated proposals as inputs, where each clip contains $2L+1$ frames. For proposals generation, we follow the previous work [8] to choose the commonly used object detector, *i.e.*, Faster R-CNN [26], to extract a fixed number of proposals with high confidence in each frame of the video. We simply denote these proposals as $\{\{r_k^n\}_{k=1}^K\}_{n=1}^N$, where the video has N frames and each frame contains K proposals. r_k^n denotes the k -th proposal in the frame n , and it is comprised of the top-left and bottom-right coordinates. The Faster R-CNN model has been trained on a large-scale detection dataset MSCOCO [45]. In our experiments, we further finetune the detector on a new dataset, *i.e.*, A2D, to improve the detection accuracy for actors.

Considering that only 3 to 5 frames are annotated in videos of A2D dataset, we sample 3 to 5 clips around the annotated frames from a video to train our model. For each clip, the middle frame, *i.e.*, the $(L+1)$ -th frame is annotated with bounding box and binary mask, while forward L frames and backward L frames are not annotated. To localize and segment the query-specified actor in consecutive frames of the clip, we first link the detected proposals $\{\{r_k^n\}_{k=1}^K\}_{n=1}^{2L+1}$ in temporal dimension and generate K tubes (*i.e.*, bounding boxes sequences). To this end, we introduce a temporal proposal aggregation mechanism that expects two proposals in adjacent frames contain the same actor or object if they possess similar spatial locations and features. Specifically, we utilize a CNN architecture to extract the appearance features in RGB stream and further obtain fixed-size (*e.g.*, 7×7) feature maps via RoIAlign operation [27] for the generated proposals. Similar to Mask R-CNN [27] which has multi-task heads for classes prediction, bounding boxes regression, and object segmentation, the small feature maps in RGB stream are fed into two different branches for actor-action matching and segmentation, respectively. For actor-action matching branch, we transform each small map into a holistic appearance representation $\mathbf{v}_k^n \in \mathbb{R}^{C_v}$ ($k \in [1, K], n \in [1, 2L+1]$) for the k -th proposal in the n -th frame via two FC layers. Similarly, we can obtain the holistic motion representation $\mathbf{f}_k^n \in \mathbb{R}^{C_v}$ ($k \in [1, K], n \in [1, 2L+1]$) for the k -th proposal in the n -th frame in Flow stream.

Next, the linking score between proposals $r_{k_1}^{n_1}$ and $r_{k_2}^{n_2}$ is formulated as:

$$\text{link}(r_{k_1}^{n_1}, r_{k_2}^{n_2}) = \text{IoU}(r_{k_1}^{n_1}, r_{k_2}^{n_2}) + \frac{\rho}{1 + ED(\mathbf{v}_{k_1}^{n_1}, \mathbf{v}_{k_2}^{n_2})}, \quad (1)$$

where the $\text{IoU}(\cdot)$ is utilized to measure the spatial similarity by computing the intersection-over-union between proposals $r_{k_1}^{n_1}$ and $r_{k_2}^{n_2}$. The features of $r_{k_1}^{n_1}$ and $r_{k_2}^{n_2}$ are denoted as $\mathbf{v}_{k_1}^{n_1}$ and $\mathbf{v}_{k_2}^{n_2}$, respectively. The $ED(\cdot)$ means the Euclidean distance which is adopted to measure the semantic similarity between features $\mathbf{v}_{k_1}^{n_1}$ and $\mathbf{v}_{k_2}^{n_2}$. The ρ is a balanced parameter. For each proposal in the n_1 -th frame, we select the proposal with maximal linking score from the n_2 -th frame. In this way, we can obtain K tubes for each video clip. Note that we only use visual features from RGB stream to calculate the semantic similarity.

To obtain the representation of each tube in the RGB stream, we apply mean pooling over features from forward L frames and backward L frames to aggregate them as temporal context of the $(L+1)$ -th frame, and perform addition with the feature of the $(L+1)$ -th frame:

$$\tilde{\mathbf{v}}_k^{L+1} = \mathbf{v}_k^{L+1} + \frac{1}{2L} \sum_{n=1; n \neq L+1}^{2L+1} \mathbf{v}_k^n. \quad (2)$$

Finally, we use one FC layer with Rectified Linear Unit (ReLU) function to transform $\tilde{\mathbf{v}}_k^{L+1}$ and obtain the appearance representation for the k -th tube. For simplicity, we denote the representation as $\mathbf{v}_k \in \mathbb{R}^{C'_v}$. In Flow stream, we apply the same way to obtain the motion representation $\mathbf{f}_k \in \mathbb{R}^{C'_v}$ for the k -th tube in the Flow clip.

B. Textual Query Representation Learning

As AASTQ takes an arbitrary textual query as input, the query may contain diversely linguistic structures or irrelevant semantics with relation to the actor and its action. To alleviate the interference of the linguistic structures and irrelevant semantics, a straightforward idea is to utilize external language parser [46], [47] or pre-defined templates [48] to parse the query into different components and then extract the actor-/action-related individual words for the actor and action localization. Nevertheless, previous works [28], [29], [49] have demonstrated that the language parser cannot work well in the query-region matching problem, because individual words lack essential contextual information for scene understanding of images. As an alternative way, Hu *et al.* [28] proposed to automatically parse the query into a triplet (*subject, relationship, object*) with a soft attention mechanism for relational reasoning. Similar to [28], Yu *et al.* [29] learned to parse the query into a triplet (*subject, location, relationship*) for object localization. In our work, we aim to explicitly model the actor and action localization from a given query. Hence, we adopt the attention mechanism to adaptively learn to attend to the words that are relevant to the actor and action, respectively.

More specifically, for a given textual query with T words $\{w_t\}_{t=1}^T$, we use Glove [50] to embed each word and obtain a vector sequence $\{\mathbf{e}_t\}_{t=1}^T \in \mathbb{R}^{C_e}$. Then a two-layer Bi-directional LSTM [51] takes the vector sequence $\{\mathbf{e}_t\}_{t=1}^T$ as input and encodes the query as follows:

$$\begin{aligned} \vec{\mathbf{h}}_t^1 &= \overrightarrow{\text{LSTM}}(\vec{\mathbf{h}}_{t-1}^1, \mathbf{e}_t); & \overleftarrow{\mathbf{h}}_t^1 &= \overleftarrow{\text{LSTM}}(\overleftarrow{\mathbf{h}}_{t+1}^1, \mathbf{e}_t), \\ \mathbf{h}_t^1 &= [\vec{\mathbf{h}}_t^1, \overleftarrow{\mathbf{h}}_t^1], \\ \vec{\mathbf{h}}_t^2 &= \overrightarrow{\text{LSTM}}(\vec{\mathbf{h}}_{t-1}^2, \mathbf{h}_t^1); & \overleftarrow{\mathbf{h}}_t^2 &= \overleftarrow{\text{LSTM}}(\overleftarrow{\mathbf{h}}_{t+1}^2, \mathbf{h}_t^1), \\ \mathbf{h}_t^2 &= [\vec{\mathbf{h}}_t^2, \overleftarrow{\mathbf{h}}_t^2]. \end{aligned} \quad (3)$$

Both directions of hidden states at each time step in the first layer are concatenated into $\mathbf{h}_t^1$ and fed into the second layer. Two directional hidden states of the second layer are concatenated into $\mathbf{h}_t^2$. Then the t -th word representation is the concatenation of $\mathbf{h}_t^1$ and $\mathbf{h}_t^2$, *i.e.*, $\mathbf{h}_t = [\mathbf{h}_t^1, \mathbf{h}_t^2] \in \mathbb{R}^{C_h}$.

($t \in [1, T]$). The encoded word representation $\mathbf{h}_t$ contains the information not only from the t -th word but also from the contextual words before and after the t -th word. Afterward, two separate FC layers followed by softmax layers are used to compute the attention weight γ_t^m ($m \in \{\text{actor}, \text{action}\}$) on each word,

$$\gamma_t^m = \frac{\exp(W_m \mathbf{h}_t)}{\sum_{i=1}^T \exp(W_m \mathbf{h}_i)}, \quad (4)$$

where the attention weight γ_t^m denotes the probability of the t -th word belong to the actor or action. Finally, the γ_t^m is applied to the embedding vector sequence $\{\mathbf{e}_t\}_{t=1}^T$ to derive the actor-related and action-related query representations,

$$\mathbf{q}^m = \sum_{t=1}^T \gamma_t^m \cdot \mathbf{e}_t. \quad (5)$$

According to the above, we adopt the attention mechanism to adaptively learn the related information about the actor and action. The learning of attention weights is weakly supervised by collaborating with the actor and action modules.

C. Actor and Action Matching in the Modular Network

To segment the actor according to descriptions about actor and its action, we first localize the tube involving the actor and its action in the video clip. We formulate this problem as a matching process in two modules, namely, the actor module and the action module. The former module is applied to localize the tube which involves the actor referred by the given query. It will compute a set of matching scores $s(t_k | \mathbf{q}^{\text{actor}})$ ($k \in [1, K]$) to measure the similarity between the actor-related query representation $\mathbf{q}^{\text{actor}}$ and the k -th tube t_k . The later module is used to localize the tube which involves the action described by the given query. This module computes a set of matching scores $s(t_k | \mathbf{q}^{\text{action}})$ ($k \in [1, K]$) to measure the similarity between the action-related query representation $\mathbf{q}^{\text{action}}$ and the k -th tube t_k . The combined matching score $s_k = s(t_k | \mathbf{q}^{\text{actor}}) + s(t_k | \mathbf{q}^{\text{action}})$ determines the similarity between the textual query q and the k -th tube t_k . Finally, the tube with the highest score is selected as the target tube which contains the actor and action referred by the textual query. The detailed design of two modules will be discussed in the following parts.

1) *The Actor Module*: As human usually tends to describe a specific actor according to appearance and location information in videos, we design the actor module to identify the query-specified actor through appearance features from RGB stream in AAMN. The actor module is shown in Fig. 3. We measure the similarity score between the actor-related query representation $\mathbf{q}^{\text{actor}}$ and feature vector $\mathbf{v}_k^{\text{actor}}$ of each tube. The tube representation captures appearance information but lacks necessary location information to distinguish those actors who have similar appearance information. Therefore, following the previous work [14], we introduce a 5-dimensional location feature $\mathbf{l}_k = [\frac{x_{tl}}{W}, \frac{y_{tl}}{H}, \frac{x_{br}}{W}, \frac{y_{br}}{H}, \frac{w \cdot h}{W \cdot H}]$ in the middle frame of the clip, where $(x_{tl}, y_{tl}, x_{br}, y_{br})$ denotes the top-left and bottom-right coordinates, (w, h) and (W, H) are the width and height of the middle proposal in the k -th tube and the video frame, respectively. We concatenate the

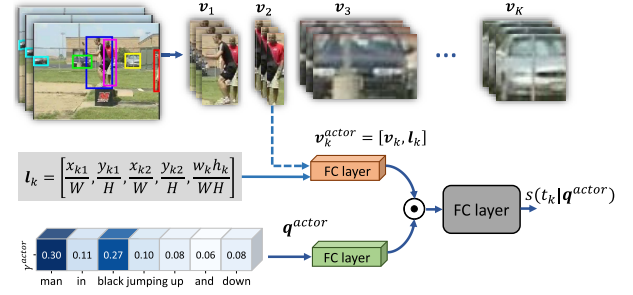


Fig. 3. Illustration of the actor module in AAMN.

location feature $\mathbf{l}_k$ with the tube representation $\mathbf{v}_k$ as the visual representation of the actor, i.e., $\mathbf{v}_k^{\text{actor}} = [\mathbf{v}_k, \mathbf{l}_k] \in \mathbb{R}^{C_v+5}$. The visual representation $\mathbf{v}_k^{\text{actor}}$ and linguistic representation $\mathbf{q}^{\text{actor}}$ are linearly transformed with FC layers and l_2 -normalized, and then perform element-wise multiplication to integrate two representations. Finally, the integrated representation is fed into one FC layer to predict an unary matching score for the actor as follows:

$$\begin{aligned} \tilde{\mathbf{v}}_k^{\text{actor}} &= W_1 \mathbf{v}_k^{\text{actor}} + \mathbf{b}_1, \\ \tilde{\mathbf{q}}^{\text{actor}} &= W_2 \mathbf{q}^{\text{actor}} + \mathbf{b}_2, \\ \mathbf{x}_k^{\text{actor}} &= \|\tilde{\mathbf{v}}_k^{\text{actor}}\|_2 \odot \|\tilde{\mathbf{q}}^{\text{actor}}\|_2, \\ s_k^{\text{actor}} &= W_3 \mathbf{x}_k^{\text{actor}} + \mathbf{b}_3, \end{aligned} \quad (6)$$

where $\odot$ denotes element-wise multiplication, $\|\cdot\|_2$ means l_2 -normalization, $\{W_1, \mathbf{b}_1, W_2, \mathbf{b}_2, W_3, \mathbf{b}_3\}$ are learnable parameters in three FC layers.

2) *The Action Module*: The action matching needs to introduce dynamic motion information as the visual cues for action perception. It is rarely investigated in previous works. Motivated by the previous approaches in [8], [10] where they introduce optical flow as dynamic motion information for action recognition, we introduce another CNN architecture, i.e., Flow stream, which takes a Flow clip as input, to learn motion information for each proposal. We obtain the holistic representation of motion information $\mathbf{f}_k$ ($k \in [1, K]$) for the k -th tube t_k with the same way as the appearance representation $\mathbf{v}_k$. However, the dynamic motion representation of each tube sometimes is not enough for action inferring. For instance, given a video with the query of “a man is standing and watching on the behind”, the actor of “man” performs actions of “standing” and “watching” without obviously spatial displacements. Thus the action cannot be shown from the dynamic motion information. Besides, the tubes with similar motion patterns also affect the result of action matching. To mitigate these problems, we devise a contextual LSTM to model the relationship between the target tube and others, then utilize the information from the contextual LSTM as auxiliary information for the action matching.

The proposed action module is shown in Fig. 4. This module contains a contextual LSTM network which assists to match the action under the ambiguous motion cues, and a gated fusion network used to weight the importance of motion information and appearance information for action matching. Concretely, the contextual LSTM takes the sequential appearance feature $\mathbf{v}_j \in \mathbb{R}^{C_v}$ ($j \in [1, K] \cap j \neq k$) of tubes as well as the whole clip feature $\mathbf{v}_0 \in \mathbb{R}^{C_v}$ as inputs. To obtain $\mathbf{v}_0$,

we take the video clip as a large tube. The feature maps of this tube from the last convolutional layer is resized into fixed sized (e.g., 7×7) via RoIAlign and then fed into two fully connected layers. We aggregate these features using the same way in Equation 2. In our model, v_0 represents global contextual information and is arranged in the first place of input sequence, i.e., $v_k^{seq} = \{v_0, v_1, \dots, v_{k-1}, v_{k+1}, \dots, v_{K-1}\}$ ($k \in [1, K-1]$). The tube features are ordered from top to down and left to right according to the proposal locations of the $(L+1)$ -th frame in the video clip. Then the sequential features v_k^{seq} are consecutively fed into LSTM to encode each tube and obtain hidden representations at each time step, i.e., $h_t^v = \text{LSTM}(h_{t-1}^v, v_t) \in \mathbb{R}^{C_c}$ ($t \in [0, K]$). Thereafter, we concatenate all hidden states from each time step $h^v = [h_0^v, h_1^v, \dots, h_{K-1}^v] \in \mathbb{R}^{K \times C_c}$ and use an attention mechanism to adaptively associate the tubes together to obtain a contextual representation $v_k^c \in \mathbb{R}^{C_c}$ for the k -th tube. The attention is calculated by:

$$\begin{aligned} y_t &= \delta(W_4 h_t^v + b_4), \\ \alpha_t &= \frac{\exp((W_5 y_t + b_5)^T)}{\sum_{j=1}^K \exp((W_5 y_j + b_5)^T)}, \\ v_k^c &= \sum_{t=1}^K \alpha_t \cdot y_t, \end{aligned} \quad (7)$$

where $\delta(\cdot)$ is ReLU function, $\{W_4, b_4, W_5, b_5\}$ are learnable parameters in FC layers.

The contextual representation v_k^c of the k -th tube is concatenated with appearance representation v_k , i.e., $\tilde{v}_k = [v_k, v_k^c] \in \mathbb{R}^{C_v + C_c}$. Then $\tilde{v}_k$ and the motion representation $f_k \in \mathbb{R}^{C_v}$ are fused with a gated way:

$$\begin{aligned} g_k^a &= W_6 \tilde{v}_k + b_6; \quad g_k^b = W_7 f_k + b_7, \\ \sigma &= \theta(W_8 [g_k^a, g_k^b] + b_8), \\ v_k^{action} &= \sigma \odot g_k^a + (1 - \sigma) \odot g_k^b, \end{aligned} \quad (8)$$

where $\theta(\cdot)$ is the sigmoid function which rescales the gate vector $\sigma \in \mathbb{R}^{C_e}$ to $[0, 1]$. Then the σ performs element-wise multiplication with sum of $g_k^a \in \mathbb{R}^{C_e}$ and $g_k^b \in \mathbb{R}^{C_e}$ for weighted fusion them. $\{W_6, b_6, W_7, b_7, W_8, b_8\}$ are learnable parameters in FC layers.

After obtaining the visual representation of action $v_k^{action} \in \mathbb{R}^{C_e}$, we compute the action matching score with the same way as the actor matching score:

$$\begin{aligned} \tilde{v}_k^{action} &= W_9 v_k^{action} + b_9, \\ \tilde{q}^{action} &= W_{10} q^{action} + b_{10}, \\ x_k^{action} &= \|\tilde{v}_k^{action}\|_2 \odot \|\tilde{q}^{action}\|_2, \\ s_k^{action} &= W_{11} x_k^{action} + b_{11}, \end{aligned} \quad (9)$$

where $\{W_9, b_9, W_{10}, b_{10}, W_{11}, b_{11}\}$ are corresponding parameters in FC layers.

D. The Actor Segmentation Network

Based on the computed matching score s_k ($i \in [1, K]$) above, we can select the target tube with the highest score, which involves the actor and action referred to by the given

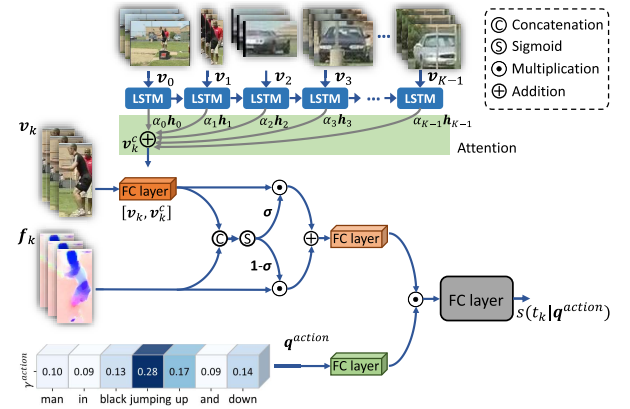


Fig. 4. Illustration of the action module in AAMN.

query. Then we take $2L + 1$ feature maps of the target tube into actor segmentation network to predict masks of the actor. The network is a small FCN which takes the fixed size, e.g., 14×14 , feature maps from RoIAlign layer as input. Similar to Mask R-CNN [27], the FCN is comprised of 4 consecutive convolution layers and 1 deconvolutional layer. The first layer consists of $256 \ 1 \times 1$ filters to fuse the appearance feature of the tube in RGB stream, and motion feature of the tube in Flow stream. The other three layers are consists of $256 \ 3 \times 3$ filters. The final deconvolutional layer up-samples features with factor 2. Since only the $(L+1)$ -th frame has mask annotation for input video clip, we choose the $(L+1)$ -th feature map of the target tube to train the actor segmentation network with input batch size as 1. During testing, we set input batch as $2L+1$ and take total feature maps of the target tube to predict the actor masks in the actor segmentation network.

E. Model Learning With Joint Actor-Action Matching and Segmentation

During training, AAMN enables joint learning for the actor-action matching and segmentation by minimizing the following combined loss function:

$$L = L_{mat} + \lambda L_{seg}, \quad (10)$$

where λ is a hyperparameter to balance the matching loss L_{mat} and the segmentation loss L_{seg} . Assuming the ground truth of the actor is contained in the k -th region proposal of the $(L+1)$ -th frame, then L_{seg} will only be computed on the k -th mask. L_{seg} is the average binary cross-entropy loss which is defined as:

$$L_{seg} = -\frac{1}{h'w'} \sum_{i=1}^{w'} \sum_{j=1}^{h'} (\eta_{ij} \log(\kappa_{ij}) + (1 - \eta_{ij}) \log(1 - \kappa_{ij})), \quad (11)$$

where h' and w' are height and width of the feature map, respectively. η is the binary ground-truth label, κ is the predicted value of the segmentation mask.

Since the triplet hinge loss has shown its strength in many cross-modal matching problem [52], [53], here we adopt it as the matching loss L_{mat} for the actor and action matching

in AAMN. L_{mat} encourages the scores of matched proposals and textual queries to be larger than those of mismatched ones:

$$L_{mat} = \sum_{q'} \max [0, \epsilon - s(p, q) + s(p, \hat{q})] + \sum_{p'} \max [0, \epsilon - s(q, p) + s(q, \hat{p})], \quad (12)$$

where p and q indicate region proposal and textual query, respectively. (p, q) and (q, p) mean the matched proposal and query pairs, and $(p, \hat{q})$ and $(q, \hat{p})$ are the mismatched pairs. $s(p, q)$ is the computed similarity between p and q . There are $K - 1$ mismatched pairs for each matched pair in our experiments. ϵ is a margin parameter between matched and mismatched pairs.

During testing, we follow previous works [19], [20], [22] to sample video clips round the annotated frames as inputs. We predict segmentation masks for the selected tube and recover predicted masks into the same resolution as the input clip. Finally, we select the mask in the middle frame (*i.e.*, annotated frame) to evaluate our model.

IV. EXPERIMENTAL RESULTS AND DISCUSSIONS

A. Datasets and Evaluation Metrics

1) *A2D Sentences*: Gavriluk *et al.* [19] augmented Actor-Action Dataset (A2D) with textual queries that describe actors and actions of interest in videos. The original A2D [5] consists of 7 actor classes which perform one of the 9 action classes. Each video provides around 3 to 5 frames sparse annotations with joint pixel-wise semantic mask and bounding box per actor-action pair. A2D sentences manually annotated 6,655 textual queries which describe actors and actions presented in videos. It is divided into 3,017 and 737 videos for training and testing. Furthermore, McIntosh *et al.* [22] annotated actors with bounding boxes in all frames of videos. The extended A2D Sentences makes it available to evaluate the full video segmentation from a textual query.

2) *J-HMDB Sentences*: J-HMDB Sentences is extended from J-HMDB [54], which is a benchmark dataset for action recognition. It comprises of 928 video clips for 21 action classes and provides pixel-wise segmentation mask for the human in action for each frame. There are 928 textual queries are annotated in J-HMDB dataset to describe what action the humans is performing in the video.

3) *Evaluation Metrics*: We adopt four types of metrics to evaluate our method as prior works [19], [22]. For single-frame video segmentation [19], each key-frame with a query is regarded as a single sample. While for the full video segmentation [22], each video with a query is considered as a single sample. The Overall Intersection-over-Union (*Overall IoU*) calculates the total intersection area between predictions and ground truth masks divided by total union area accumulated over all test samples, which tends to favor large actors and objects. The *Mean IoU*, which treats large and small actors or objects equally, is calculated as the average over the *IoU* of each testing samples. The precision at different threshold $P@X$ ($X \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$), which is the percentage of testing samples whose *IoU* higher than specific threshold. The mean average precision (*mAP*) over $[0.50 : 0.05 : 0.95]$ is also computed to measure the segmentation performance.

B. Implementation Details

In AAMN implementation, we choose Faster R-CNN [26] with VGG-16 network as two CNN architectures to extract fixed-size proposal features with RoIAlign layer [27]. AAMN takes a textual query, a RGB clip and a Flow clip as well as pre-generated proposals as inputs. The frames of the video clip are resized and padded to 512×512 as previous works [19], [20]. The optical flow is a tensor of three channels with x and y coordinates of the flow as well as the flow magnitude [8], [10]. Since the generation of high-quality proposals is important for actor and action localization, we choose Faster R-CNN [26] with a backbone of ResNet-101 as an external object detector. This model is trained on MSCOCO dataset [45] and further finetuned on A2D [19]. In our experiments, the dimension of C_v is 4096, the dimension of C'_v is set as 1024. For the textual query encoding, we adopt two-layer Bi-directional LSTM (Bi-LSTM). The dimension of the word embeddings that are input into Bi-LSTM is $C_e = 300$, and the forward and backward hidden states from Bi-LSTM are $C_h = 512$. In order to obtain fixed dimensional query representation, long queries are truncated and short queries are padded with zeros. The fixed length of the query is set to 20 in our experiments. Besides, the dimension of hidden states in the contextual LSTM is $C_c = 512$.

We implement our proposed model with Tensorflow. Two CNN architectures are initialized with pre-trained weights on MSCOCO dataset [45]. During the model training, we use stochastic gradient descent (SGD) with initial learning rate of 0.0001, momentum of 0.95, weight decay of 0.0005. The learning rate decreases by 10 times at 1.0×10^5 iterations and the model is trained for 1.5×10^5 iterations. The balanced parameter ρ in Equation 1 is selected as 1.0. The margin ϵ is set to 0.1 in the matching loss L_{mat} in Equation 12.

C. Ablation Studies

1) *Selection of Proposal Number*: The selection of proposal number is critical to model visual region context in the contextual LSTM of the action module. Thus we conduct experiments with various numbers of proposals to analyze the impact of proposals number in our model. We first extract a set of proposals per frame with high confidence utilizing an object detector. Then we select a specific number of proposals and feed them into our model. In our experiments, the number is set to increase from 2 to 12 with an interval of 2. Before evaluating the segmentation results with different proposals number, we first analyze the actor and action localization results with bounding boxes. Here, the precision of localization $P@X$ ($X \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$) is evaluated by calculating the IoU ratio between the true bounding box and the top predicted box in annotated frames. If the IoU is larger than the threshold, the prediction is considered as a true positive, otherwise it is counted as a false positive. The total true positive counts are then averaged over all testing samples to obtain $P@X$. Table I shows the precision of localization at various thresholds with various numbers of proposals. It can be seen that the precision increases with increment of proposals number. When the number reaches a certain value, *i.e.*, $K = 6$,

TABLE I

ANALYSIS OF LOCALIZATION PRECISION AT DIFFERENT THRESHOLD WITH DIFFERENT PROPOSALS NUMBER IN EACH VIDEO FRAME

Models	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9
K = 2	67.5	63.8	58.7	48.5	20.1
K = 4	69.1	65.2	59.6	49.1	20.7
K = 6	69.0	66.1	60.2	48.7	21.5
K = 8	68.2	65.5	59.7	48.1	20.9
K = 10	67.9	65.0	59.4	47.9	21.0
K = 12	67.2	63.5	57.9	46.5	19.3

TABLE II

THE ANALYSIS OF SEGMENTATION RESULTS WITH DIFFERENT PROPOSAL NUMBER

Models	Overlap					mAP		IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean	
K = 2	66.9	60.6	50.7	26.5	1.7	36.8	59.9	53.6	
K = 4	68.7	64.6	53.6	30.4	3.1	39.5	62.9	55.7	
K = 6	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6	
K = 8	67.7	63.2	53.1	31.6	2.9	40.6	62.7	54.8	
K = 10	67.1	62.9	52.6	29.3	2.2	39.9	60.9	53.3	
K = 12	66.6	61.8	50.5	27.8	2.1	37.2	58.8	52.7	

TABLE III

THE IMPACT OF INPUT FRAME NUMBER

Models	Overlap					mAP		IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean	
L = 0	68.1	62.9	52.3	29.4	2.8	39.6	61.7	55.2	
L = 2	68.4	64.1	53.2	30.1	2.9	39.9	62.7	55.3	
L = 4	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6	
L = 8	68.9	64.4	54.2	31.9	3.2	40.5	62.8	55.5	

the precision tends to decline in most metrics. This reveals that the contextual LSTM progressively encodes local features into a discriminate vector representation which contains more details of video content, but too more proposals may bring noises. The segmentation results with different number of proposals are shown in Table II. Since our model is a two-stage method, the segmentation performance is closely relevant to the localization performance, the best segmentation performance is achieved when the number of proposals equals 6. Thus we set the number of proposals as 6 in our model.

2) *Impact of Input Frame Number*: The AAMN takes RGB and Flow clips as inputs, where each clip contains $2L + 1$ frames. To investigate the impact of frame number in the clip, we perform experiments with different L , *i.e.*, $L = \{0, 2, 4, 8\}$, the corresponding frames are $\{1, 5, 9, 17\}$ in a clip. The experimental results are shown in Table III. We observe that the single frame input, *i.e.*, $L = 0$, has the worst performance in all metrics. By increasing the number of input frames, the performance is increased, especially in $P@0.6$ and $P@0.7$. It demonstrates that the aggregation of temporally contextual information is helpful for actor and action matching in two modules, and leads to the improvement of actor segmentation. The best results are achieved under most metrics when the frame number is 9. The performance drops by further increasing the frame number. The possible reason is that proposals of the actor or object cannot link well by computing similarity of spatial locations over a long frames sequence, irrelevantly temporal information is aggregated into the annotated frame. Therefore, we select the frame number as 9, *i.e.*, $L = 4$, in our experiments.

3) *Study of Ablation Models*: To systematically evaluate the relative contributions of different components in AAMN, we design seven ablation models as shown in Lines 1-7 of Table IV. a) “Baseline Model” utilizes concatenation of the appearance feature v_i and the motion feature f_i of the i -th tube as the visual representation, the mean pooling of embedded word sequence as a language representation. The matching score is computed using the same way as the actor matching score. b) “w/o Actor Module” means that AAMN only contains the action module. c) “w/o Actor Module” means that AAMN only contains the action module. d) “w/o Location Feature” indicates that the actor module without the location feature l_i . e) “w/o Contextual LSTM” indicates that the action module without the contextual LSTM. f) “w/o Attention” refers to using element-wise mean pooling over the concatenated hidden states h^v in the contextual LSTM to obtain the contextual representation v_i^c . g) “w/o Gated Fusion” refers to using concatenation instead of gated fusion to merge the appearance feature $\tilde{v}_i$ and the motion feature f_i in the action module. As a comparison, the “Full Model” is shown in the last Line of Table IV.

From the experimental results in Lines 1-7 and Line 14 of Table IV, we can observe that: i) Benefiting from the modular design, the “Full Model” outperforms the “Baseline Model” by a large margin in terms of different metrics. ii) The performance of individual modules is close to or slightly worse than the performance of “Baseline Model”, while the combination of them can significantly improve the performance. This demonstrates that two modules can collaborate and facilitate each other for the actor and action matching. iii) Modeling the contextual information using the contextual LSTM with attention is helpful to promote the action matching and further improve the segmentation performance. iv) Compared with the concatenation way to fuse the appearance feature $\tilde{v}_i$ and the motion feature f_i , the gated fusion way can learn to weight the importance of the motion feature f_i and the appearance feature $\tilde{v}_i$ for accurately matching the described action in a textual query.

4) *Impact of Optical Flow on Different Components*: The impact of optical flow have been studied when we add optical flow features in different components of AAMN, the corresponding results are shown in Lines 8-13 of Table IV. a) “w/o Flow Stream” denotes that AAMN only contains RGB stream and the motion feature f_i is replaced with the appearance feature v_i in action module. b) “w/o RGB Stream” means that AAMN only contains flow stream and the appearance feature v_i is replaced with the motion feature f_i in two modules. c) “(FCN) w/o Optical Flow” indicates that AAMN predicts the segmentation mask from the appearance features v_i . d) “(Action Module) w/o Optical Flow” denotes that AAMN only uses the appearance feature $\tilde{v}_i$ for action matching in the action module. e) “(Action Module) w/o RGB” means that AAMN only utilizes the motion feature f_i for action matching in the action module. f) “(Actor Module) w/ Optical Flow” refers to adding the motion feature f_i into the actor module.

From the experimental results in Lines 8-14 of Table IV, we can conclude that: i) The two-stream framework performs better than single stream framework, because the appearance

TABLE IV
ABLATION STUDY ON A2D SENTENCES

Models	Overlap					mAP	IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean
Baseline Model	63.3	56.7	39.5	17.2	1.1	31.9	55.2	48.5
AAMN w/o Actor Module	48.0	44.3	36.8	20.7	1.9	27.8	45.7	39.7
AAMN w/o Action Module	63.7	58.7	42.5	22.3	1.4	33.7	58.9	51.8
AAMN w/o Location Feature	68.2	63.7	53.4	31.8	2.7	39.5	62.4	54.7
AAMN w/o Contextual LSTM	67.9	62.5	51.8	27.8	2.0	37.8	60.3	53.6
AAMN w/o Attention	68.3	63.4	52.8	31.7	2.1	39.9	61.5	54.8
AAMN w/o Gated Fusion	68.1	62.7	52.6	29.7	2.2	38.9	61.7	53.8
AAMN w/o Flow Stream	65.2	58.0	44.5	20.8	1.6	34.8	58.8	51.3
AAMN w/o RGB Stream	55.1	43.3	22.7	4.5	0.1	22.2	46.5	42.0
AAMN (FCN) w/o Optical Flow	67.5	61.9	49.1	26.4	2.0	38.0	61.8	53.9
AAMN (Action Module) w/o Optical Flow	67.1	62.2	51.6	29.3	1.6	38.0	60.9	53.3
AAMN (Action Module) w/o RGB	65.9	61.7	52.2	31.4	1.4	37.8	59.8	52.5
AAMN (Actor Module) w/ Optical Flow	68.0	63.1	53.0	30.0	2.4	39.0	61.7	54.3
AAMN (Full Model)	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6

TABLE V
THE IMPACT OF OPTICAL FLOW ON THE SEMANTIC SIMILARITY

Features	Overlap					mAP	IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean
Flow	67.7	62.0	49.4	23.7	0.8	37.4	61.1	53.5
RGB	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6
Flow+RGB	68.4	64.0	54.0	32.4	2.6	40.7	62.0	55.2

feature v_i is essential for actor matching and the motion feature f_i plays a key role for action matching. ii) The combination of the motion feature f_i and the appearance feature v_i can improve the segmentation performance in AAMN. iii) The individual appearance feature $\tilde{v}_i$ or motion feature f_i performs worse than their combination for action matching, since the appearance feature $\tilde{v}_i$ encoded region contextual information can distinguish those actions which have ambiguous motion cues. iv) The motion feature f_i cannot promote the actor matching, this may be because the input of two modules are same, the modular network cannot learn discriminative information for the actor and action matching.

We also investigate the impact of optical flow on the semantic similarity of linking score in Equation 1. Specifically, we generate tubes by calculating the semantic similarity using visual features from Flow stream, RGB stream, and combination of two streams. Experimental results are summarized in Table V. It can be observed that our model achieves the best performance by calculating the semantic similarity only using visual features from RGB stream, while the performance degrades obviously using visual features from Flow stream or combination of two streams. This is reasonable because we expect to link objects in consecutive frames by capturing invariant features about these objects. Specifically, visual features from RGB stream learn appearance information about objects. Such information contains visual cues about categories, colors, and shapes of objects. Thus, appearance information is usually invariant between adjacent frames. However, visual features from Flow stream mainly learn dynamic motion information performed by objects. For each object, the learned information is not stable in single frames for different actions, view angles, movement speeds, and occlusions. Lots of works [55] usually aggregate multiple frames of visual features from Flow stream for action recognition. Calculating the semantic similarity with such dynamic

motion information is not accurate to link objects. Therefore, we only use visual features from RGB stream to calculate the semantic similarity in our work.

5) *Impact of Temporal Aggregation Strategy*: Due to sparse annotations of A2D dataset, we generate the tube representation by aggregating visual features from adjacent frames into the annotated frame (*i.e.*, the middle frame) to train our model. Except the temporal aggregation strategy in Equation 2, we also explored another aggregation strategy by directly averaging visual features over all frames of the clip. Experimental results are summarized in Table VI. It can be observed that the strategy in Equation 2 obtains better results than averaging visual features over all frames in most metrics. This is because visual features from the annotated frame are more discriminative for actor and action matching, and assigning larger weights to visual features from annotated frame can improve the localization precision of the actor.

6) *Analysis of Learning Strategy*: Our AAMN is a two-stage method which first localizes the actor/action-related tube and then performs segmentation within the localized tube. Thus we can choose two learning strategies for AAMN: jointly learn actor-action matching and segmentation at same time, or separate learn them one-by-one. To compare with our main joint learning strategy, we perform two separate learning experiments, *i.e.*, “M + S” and “S + M”. The “M + S” means we first train the matching branch until convergence and then train the FCN branch for the actor segmentation. The “S + M” denotes we first train the FCN branch until convergence and then train the matching branch. The experimental results with different learning strategies are shown in Table VII. Note that the actor-action matching and segmentation branches share the same backbone of feature extractor, therefore when separately trained, the final AAMN may be bias to the actor-action matching or segmentation. By comparing the results in Table VII,

TABLE VI
COMPARISON OF TWO TEMPORAL AGGREGATION STRATEGIES

Models	Overlap					mAP	IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean
Average	69.1	63.4	53.0	30.4	2.8	40.7	62.8	55.7
Ours	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6

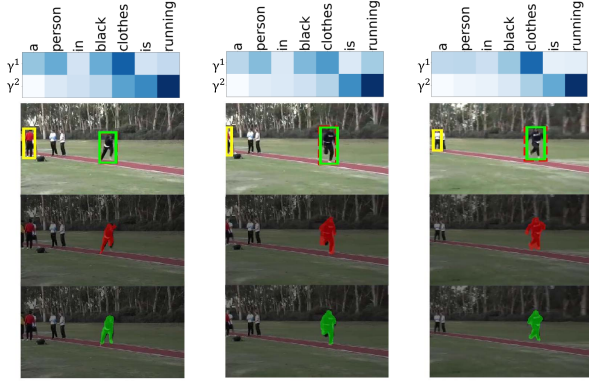
TABLE VII
THE ANALYSIS OF DIFFERENT TRAINING STRATEGIES

Models	Overlap					mAP		IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean	
M + S	66.2	55.2	34.3	9.3	0.4	29.8	55.7	50.0	
S + M	65.4	58.5	41.9	15.9	1.6	33.1	57.9	50.8	
Joint	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6	

TABLE VIII
THE ANALYSIS OF HYPER-PARAMETER λ

λ	Overlap					mAP		IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean	
0.01	59.9	44.1	20.2	2.9	0	21.9	51.8	46.3	
0.1	64.8	54.5	35.2	10.5	0.7	29.7	56.9	49.7	
1.0	67.0	60.7	45.0	21.8	1.2	35.7	59.5	52.9	
2.5	67.7	61.6	48.6	25.0	2.5	37.5	60.7	53.9	
5.0	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6	
7.5	68.9	64.1	53.8	30.6	2.3	40.5	63.0	55.6	
10.0	68.2	63.2	53.3	30.6	2.2	40.2	62.0	55.1	
12.5	68.3	63.5	53.1	28.9	1.6	38.8	61.3	54.2	
15.0	67.4	62.4	51.3	28.0	1.9	37.7	60.1	53.5	

(a) Q : a person in black clothes is running



(b) Q : a man with white shirt is throwing a ball

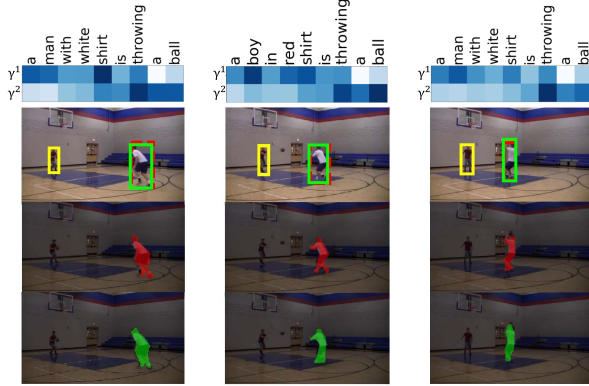


Fig. 5. Visualization of word attention weights and segmentation results on A2D Sentences. For each example, we sample three annotated frames from a video. The first row shows the learned attention weights for the textual query, the second row shows the localization results in annotated frames, the third row shows the ground truth masks (red), the last row shows the predicted masks (green) for the referred actor.

we can conclude that joint learning is helpful to avoid subtask biases and achieves best performance.

7) *Analysis of Hyper-Parameter*: In Equation 10, a hyper-parameter λ is introduced to balance the matching loss L_{mat} and the segmentation loss L_{seg} in joint learning process. $\lambda = 1.0$ means L_{mat} plays an equally important role as L_{seg} . Here we conduct experiments to analyze the impacts of λ . We vary λ

TABLE IX
COMPARISON WITH RANDOM ATTENTION WEIGHTS

Weights	Overlap					mAP		IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean	
Random	65.2	60.0	49.2	26.8	1.3	36.5	58.3	51.8	
Prediction (Ours)	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6	

from 0.001 to 15.0 in experiments and present corresponding results in Table VIII. It can be found that the segmentation performance is improved with increasing of λ , and reaches the best performance at most evaluation metrics when $\lambda = 5.0$. After that, the performance tends to descend when λ keeps increasing.

8) *Analysis of Word Attention Weights*: As the lack of word-level annotations, the language attention learning is weakly supervised by collaborating with the actor and action modules. According to the role of each word plays in two modules, the related information about the actor and action is adaptively learned from the textual query. Since it is hard to quantitatively evaluate the accuracy of learned word attention weights, we visualize word attention weights and corresponding segmentation results on key frames of two video from A2D Sentences in Fig. 5.

In Fig. 5, the first row shows the visualization of learned word attention weights for the textual query, where γ^1 and γ^2 denote word attention weights corresponding to the actor module and action module, respectively. The darker color indicates larger value of weight. We can observe that the language attention model enables to attend to right words that are relevant to the actor and action respectively. Specifically, γ^1 has higher values on those word that are nouns and attribute words referring to the actor. While γ^2 obtains higher values on those words that are verbs or verb-related phrases. In the second row, it is obvious that our model can accurately localize the actor with green bounding boxes in different frames. This can be attributed to the modular design in our model. The predicted bounding box from the detector is very close to the ground truth bounding box (red dashed line). The yellow bounding box is the most relevant contextual region which has the highest attention weight in the contextual LSTM of the action module. The third row shows the ground truth segmentation masks of the actor in key frames. The last row shows the predicted segmentation mask of the referred actor with our AAMN. We can observe that our model is able to predict accurate binary masks for the referred actor in key frames.

To further demonstrate our model can effectively capture related information about the actor and action from the textual query, we conduct an additional experiment by randomly setting the word attention weights for the textual query during the model testing process. The experimental results are shown in Table IX. It can be seen that the segmentation performance deteriorates notably by assigning random attention weights for the textual query. This indicates that the language attention learning enables to capture key-word information for the actor matching and action matching in AAMN. In Fig. 6 (b) and (c), we present two testing examples with predicted attention weights and random attention weights,

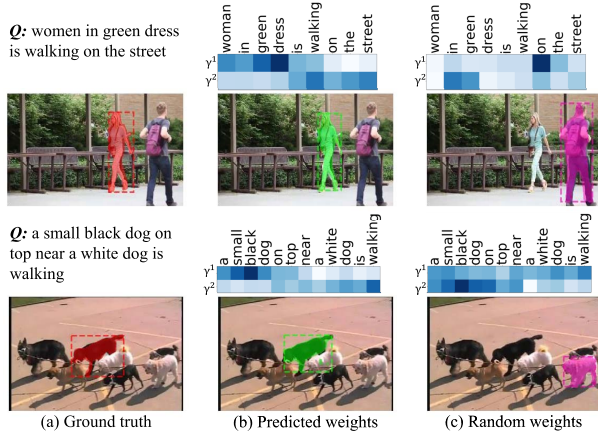


Fig. 6. Visualization of experimental results with predicted and random attention weights. (a) The ground truth. (b) Segmentation results with predicted word attention weights. (c) Segmentation results with random word attention weights.

respectively. It can be observed that the predicted attention weights focus on key words about the actor and action, and thus the model can correctly localize and segment the referred actor. However, the model with random attention weights fails to localize the referred actor, as the model ignores meaningful information about the actor and action from the textual query.

D. Comparison With State-of-the-Art Methods

1) *Single-Frame Segmentation From a Textual Query*: Since sparse annotations on A2D dataset, previous approaches [19]–[22] take a video clip around the annotated frame and a textual query as inputs. They utilized pre-trained 3D CNN to extract the clip features, and conduct temporal average pooling to obtain a 2D feature map. The segmentation performance is evaluated on the annotated frame by upsampling the fused multi-modal features from the low resolution to high resolution by introducing 2D deconvolutional layers. While other approaches [15], [56] are proposed for image segmentation from a textual query, they directly take the annotated frame and a textual query as inputs and ignore the temporal modeling. Therefore, aforementioned approaches are single-frame segmentation models as they only output single-frame predictions at a time. Different from them, AAMN is a multi-frame model which outputs multi-frame predictions at a time. Specifically, by taking a video clip and a textual query as inputs, AAMN first localizes the tube that contains the referred actor and action in the video clip, and then feeds the tube into a FCN. The FCN is only trained with the annotated frame, *i.e.*, middle frame, of the selected tube during the model learning. The FCN can predict all masks of the referred actor along the selected tube during the model testing. In this set of experiments, we present the segmentation results of AAMN on annotated frames as previous works.

In Table X, we present comparisons of our model with previous methods on A2D Sentences. For fair comparison with methods that only take the RGB clip as input, we remove the Flow stream and the gated fusion in the action module. The experimental results are shown in Line 7 of Table X. It can

be observed that our method outperforms previous methods under most metrics, especially for the *Overlap* in $P@0.5$, $P@0.6$, and $P@0.7$, which surpass the best results by 4.5%, 5.5%, and 4.0%, respectively. Previous works [57]–[59] have demonstrated that integrating the optical flow is beneficial to improve the performance of video object segmentation. The methods [19], [20] also explored two-stream models that take RGB and Flow clips as inputs. Their methods fused RGB and Flow streams by computing a weighted average of response maps from each stream. We also extend the single-stream model in [21] to a two-stream model by adopting the same way to fuse RGB and Flow streams. Our method is a two stream model that takes RGB and Flow clips as inputs. The optical flow is introduced to localize the action which performs by the referred actor. The experimental results of our full model are shown in Line 11 of Table X. We can see that the performance of our model significantly outperforms state-of-the-art results from single-stream models, and our model achieves the best performances compared with previous two-stream models. Specifically, compared with the best results from two-stream model in [21], our model brings 5.6%, 9.2%, 11.7% and 6.7% improvements under metrics of $P@0.5$, $P@0.6$, $P@0.7$, and $P@0.8$. The performance under metrics of *mAP*, *Overall IoU*, and *Mean IoU* also achieves 7.0%, 0.5%, and 1.8% improvements, respectively.

We note that the performance of our model is slightly worse than the best results for the *Overlap* in $P@0.9$, and the performance improvement is small in *Overall IoU*. The main reason is that we adopt proposal-based approach [27] to predict the mask of actor on a 28×28 grid irrespective of actor size. Such a grid is sufficient for small actors, but for large actors further upsampling with bilinear interpolation will over-smooth the fine-grained details [60]. While previous works [19]–[22] predict the mask of the actor by gradually upsampling the full feature map with multiple deconvolutional layers. They introduce skip-connections in the model [22] or supervisions on different resolutions during model training [19], [20]. Therefore, these methods tend to preserve fine-grained details and possess higher overlaps for both small and large actors.

In addition, we also compare the inference speed for different methods in Table X. The inference speed indicates the total time for the method predicts the final segmentation masks from a clip-query pair. It can be observed that the inference speed of our method is slower than other methods. This is because our method requires more time for proposal detection per frame in the input video clip. However, our method provides a new solution for text-based video segmentation from another perspective, which is significantly different from existing methods. And our method is more efficient than other methods, as they only predict one-frame segmentation results from the clip-query pair, while our method can segment all frames of the video clip.

Following [19], we also evaluate our method on J-HMDB Sentences to validate the generalization ability of our method. In our experiments, the testing model is pre-trained on A2D Sentences, and J-HMDB Sentences is used as a testing dataset. For fair comparisons, we also uniformly sample three clips

TABLE X
COMPARISON WITH THE STATE-OF-THE-ART METHODS ON A2D SENTENCES

Methods	Input	Overlap					mAP	IoU		Speed
		P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean	
Hu <i>et al.</i> [15]	RGB	34.8	23.6	13.3	3.3	0.1	13.2	47.4	35.0	60.6 ms
Li <i>et al.</i> [56]	RGB	38.7	29.0	17.5	6.6	0.1	16.3	51.5	35.4	-
Gavrilyuk <i>et al.</i> [19]	RGB	47.5	34.7	21.1	8.0	0.2	19.8	53.6	42.1	95.6 ms
ACGA [20]	RGB	55.7	45.9	31.9	16.0	2.0	27.4	60.1	49.0	108.9 ms
CMDy [21]	RGB	60.7	52.5	40.5	23.5	4.5	33.3	62.3	53.1	125.3 ms
VT-Capsule [22]	RGB	52.6	45.0	34.5	20.7	3.6	30.3	56.8	46.0	-
AAMN (Ours)	RGB	65.2	58.0	44.5	20.8	1.6	34.8	58.8	51.3	446.6 ms
Gavrilyuk <i>et al.</i> [19]	RGB+Flow	50.0	37.6	23.1	9.4	0.4	21.5	55.1	42.6	174.7 ms
ACGA [20]	RGB+Flow	-	-	-	-	-	28.7	60.6	50.3	184.8 ms
CMDy [21]	RGB+Flow	63.3	55.0	42.8	25.7	4.9	34.2	62.9	53.8	201.3 ms
AAMN (Ours)	RGB+Flow	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6	546.5 ms

TABLE XI
COMPARISON WITH THE STATE-OF-THE-ART METHODS ON J-HMDB SENTENCES

Methods	Input	Overlap					mAP		IoU	
		P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean	
Hu <i>et al.</i> [15]	RGB	63.3	35.0	8.5	0.2	0.0	17.8	54.6	52.8	
Li <i>et al.</i> [56]	RGB	57.8	33.5	10.3	0.6	0.0	17.3	52.9	49.1	
ACGA [20]	RGB	75.6	56.4	28.7	3.4	0.0	28.9	57.6	58.4	
CMDy [21]	RGB	74.2	58.7	31.6	4.7	0.0	30.1	55.4	57.6	
VT-Capsule [22]	RGB	67.7	51.3	28.3	5.1	0.0	26.1	53.5	55.0	
AAMN (Ours)	RGB	74.6	57.3	25.6	1.5	0.0	27.8	55.6	56.3	
Gavrilyuk <i>et al.</i> [19]	RGB+Flow	69.9	46.0	17.3	1.4	0.0	23.3	54.1	54.2	
ACGA [20]	RGB+Flow	-	-	-	-	-	29.5	57.9	59.1	
CMDy [21]	RGB+Flow	76.6	60.0	32.9	5.0	0.0	30.8	56.2	58.0	
AAMN (Ours)	RGB+Flow	78.5	63.4	34.6	3.8	0.0	32.0	59.3	59.0	

for each video as previous works. The experimental results are shown in Table XI. In Line 6 of Table XI, we show experimental results that we remove the Flow stream in our model. It can be observed that our method achieves superior results compared with previous methods. In Line 10 of Table XI, we show the experimental results from full model which takes RGB and Flow clips as inputs. Compared with two-stream models, our method achieves state-of-the-art performance under most metrics. Specifically, compared with the best results from two-stream method in [21], our method achieves 1.9%, 3.4% and 1.7% improvements in terms of $P@0.5$, $P@0.6$ and $P@0.7$. The mAP and $Overall IoU$ also surpass the best results by 1.2% and 1.4%, respectively. The results in terms of $P@0.8$ and $Mean IoU$ are slightly worse than the best results. It is probably caused by the backbone that cannot extract fine representations without finetuning on J-HMDB Sentences.

To validate the importance of temporal modeling by associating objects cross frames for text-based video segmentation, we also compare our method with some state-of-the-art referring expression segmentation methods on A2D Sentences. The experimental results are shown in Table XII. It can be observed that our method achieves superior performance than these methods. This demonstrates that the temporal modeling by associating objects cross frames is truly helpful to localize and segment the referred actor in the video.

2) *Full Segmentation From a Textual Query*: Previous methods [19]–[21] are very time consuming for full video segmentation as they only predict one frame segmentation at a time. McIntosh *et al.* [22] further replaced 2D deconvolutional layers with 3D deconvolutional layers in their decoder network, which enables the model to predict multi-frame segmentation results at a time. Similar to [22], our method

TABLE XII
COMPARISON WITH STATE-OF-THE-ART REFERRING EXPRESSION SEGMENTATION METHODS

Models	Overlap					mAP		IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean	
RRN [38]	42.0	35.5	27.5	15.4	2.5	22.4	56.4	37.3	
CMSA [41]	46.7	38.5	27.9	13.6	1.7	-	59.2	40.5	
BRINet [42]	55.7	46.4	33.0	17.5	3.3	28.3	63.0	47.4	
CMPC [43]	59.0	52.7	43.4	28.4	6.8	35.1	64.9	51.5	
Ours	68.9	64.2	54.5	32.4	3.4	41.2	63.4	55.6	

TABLE XIII
EXPERIMENTAL RESULTS ON A2D SENTENCES FOR FULL VIDEO SEGMENTATION

Methods	Overlap					mAP		IoU	
	P@0.5	P@0.6	P@0.7	P@0.8	P@0.9	0.5:0.95	Overall	Mean	
VT-Capsule (pixel)	9.6	1.6	0.4	0.0	0.0	1.8	34.4	26.6	
AAMN (pixel)	13.9	2.8	0.3	0.0	0.0	2.5	35.6	29.8	
VT-Capsule (bbox)	41.9	33.3	22.2	10.0	0.1	21.2	51.5	41.3	
AAMN (bbox)	46.7	37.8	24.0	9.1	0.2	21.5	52.1	43.5	

is a multi-frame segmentation model, which can be applied to full video segmentation by segmenting video clip-by-clip.

To explore full video segmentation, McIntosh *et al.* further annotated actors with bounding box in all frames of the video. They utilized bounding boxes to train and evaluate their model. The predictions of the model is block-like, which is not precise enough for pixel-level segmentation of the actor. Different from them, we only train our model with ground truth masks in key frames. For ease of comparison with [22], we also evaluate our model with bounding box annotations in all frames of the video. The experimental results are shown in Table XIII. The performance is evaluated with pixel-wise segmentation output in the first and second lines. In the



Fig. 7. The qualitative results shown in sampled frames for several videos from A2D Sentences (Best viewed in color). For each example, from the first row to the fourth row show the sampled video frames, the segmentation results from [19], the segmentation results from [20], and the segmentation results from our AAMN. The queries are given on the top and their colors correspond to mask colors.

third and last lines, the bounding box is placed around the fine-grained segmentation masks for evaluation. As observed from Table XIII, our method achieves superior performance compared with previous method in [22] for the full video segmentation.

E. Visualization and Failure Cases

1) *Qualitative Results:* To qualitatively validate the effectiveness of our method, we provide some segmentation results in sampled frames for two videos on A2D Sentences. The visualization results are shown in Fig. 7, where mask colors correspond to query colors. For each example, the first row shows input video frames. The second row shows the results produced by the method in [19]. The third row shows the results produced by the method in [20]. The last row shows the results produced by our method. It can be observed that our method can produce better visual qualitative results than previous methods, and more details of the actor are presented in our results. For example, the boundaries of the toddler in the first video and two man in the second video are clearer than the results produced by two other methods. Besides, segmentation results from methods in [19], [20] involve confused pixels between different actors in two videos. Such confused pixels indicate that these methods cannot precisely distinguish the referred actors in sampled frames. Different from them, our method enables to correctly distinguish the referred actors and generate more precise segmentation masks.

2) *Failure Cases:* Despite our AAMN has achieved significant improvements than other methods, there are still some challenging scenarios that cannot be handled well. Some typical failure cases from A2D Sentences are shown in Fig. 8. We argue that these failure cases are mainly due to the following three aspects. *First*, the ambiguous description for the target actor. For example, in the first video, it aims to segment

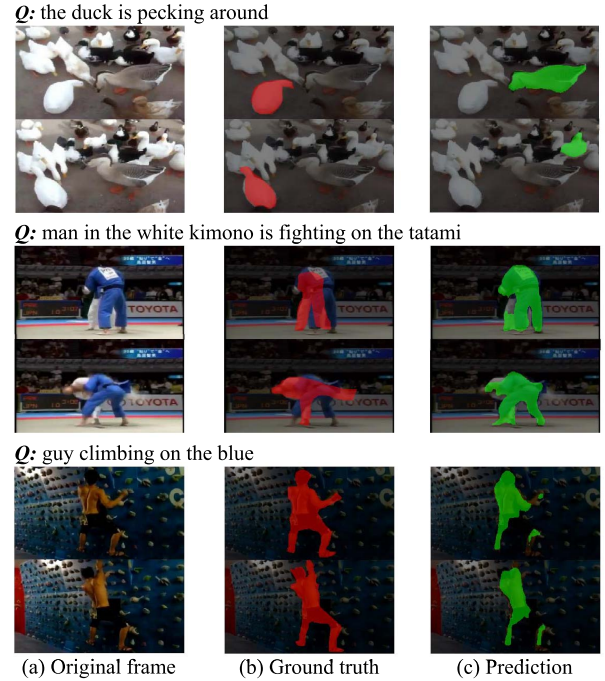


Fig. 8. Some failure cases on A2D Sentences. (a) The original video frame. (b) The ground truth. (c) The failures of prediction caused by the ambiguous description for the target actor (the first video), the occlusion between different instances (the second video), and the blurry boundary of the target actor (the third video). Note that we sample two annotated frames from each video.

the duck on the bottom-left with the textual query “*the duck is pecking around*”. Our model suffers from difficulty to identify the referred duck and ducks around it, because they have similar appearance and motion information. *Second*, the occlusion between different instances. The occlusion in the video makes it difficult to correctly localize the target actor with inaccurate detection results. It also deteriorates the segmentation results as the model cannot distinguish different instances within the localized region. For example, in the second video, our model fails to segment the man in white as he is partially occluded by the man in blue. *Third*, the blurry boundary of the target actor. Our model fails to produce sharper predictions around actor boundaries when the actor involves similar colors with background or fast motion in the video. For example, in the third video, our predictions cannot precisely segment the right thigh of the man because the part of right thigh is hard to be distinguished from background. Some of failure cases caused by such blurry boundary may be improved by enhancing shape information from adjacent frames.

V. CONCLUSION

In this paper, we have studied a challenging problem which is text-based video segmentation. To deal with this problem, we proposed an actor and action modular network to achieve video-query symmetrical matching. We first generate actor-/action-related tubes for input video clip, and learn the actor-/action-related language representations from the textual query. Then we use the modular network to locate the target tube, which involves the referred actor and action. Finally, the target tube is fed into a tiny FCN to predict the binary masks of the actor within the tube. We conducted extensive

experiments to demonstrate the effectiveness of our method, and our method achieves state-of-the-art performance on two benchmark datasets. To explore full video segmentation, McIntosh *et al.* annotated actors with bounding boxes in all frames of the video. But the segmentation results are not precise enough for the model trained with bounding boxes. Considering pixel-level annotation for all frames of video is time-consuming and cost expensive, we will explore weakly supervised text-based video segmentation in the future work.

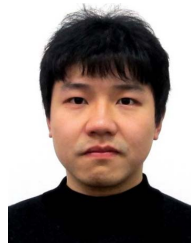
REFERENCES

- [1] J. C. Niebles and L. Fei-Fei, "A hierarchical model of shape and appearance for human action classification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2007, pp. 1–8.
- [2] R. Girdhar, D. Ramanan, A. Gupta, J. Sivic, and B. Russell, "ActionVLAD: Learning spatio-temporal aggregation for action classification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2017, pp. 3165–3174.
- [3] A. Klaser, M. Marszalek, C. Schmid, and A. Zisserman, "Human focused action localization in video," in *Proc. ECCV*, vol. 6553, 2010, pp. 219–233.
- [4] Z. Shou, D. Wang, and S.-F. Chang, "Temporal action localization in untrimmed videos via multi-stage CNNs," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 1049–1058.
- [5] C. Xu, S.-H. Hsieh, C. Xiong, and J. J. Corso, "Can humans fly? Action understanding with multiple classes of actors," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 2264–2273.
- [6] C. Xu and J. J. Corso, "Actor-action semantic segmentation with grouping process models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 3083–3092.
- [7] Z. Qiu, T. Yao, and T. Mei, "Learning deep spatio-temporal dependence for semantic video segmentation," *IEEE Trans. Multimedia*, vol. 20, no. 4, pp. 939–949, Apr. 2018.
- [8] V. Kalogeiton, P. Weinzaepfel, V. Ferrari, and C. Schmid, "Joint learning of object and action detectors," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 4163–4172.
- [9] K. Dang, C. Zhou, Z. Tu, M. Hoy, J. Dauwels, and J. Yuan, "Actor-action semantic segmentation with region masks," 2018, [arXiv:1807.08430](https://arxiv.org/abs/1807.08430).
- [10] J. Ji, S. Buch, A. Soto, and J. C. Niebles, "End-to-end joint semantic segmentation of actors and actions in video," in *Proc. ECCV*, 2018, pp. 702–717.
- [11] A. J. Rana and Y. S. Rawat, "We don't need thousand proposals: Single shot actor-action detection in videos," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 2960–2969.
- [12] Y. Yan, C. Xu, D. Cai, and J. J. Corso, "Weakly supervised actor-action segmentation via robust multi-task ranking," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1298–1307.
- [13] J. Chen, Z. Li, J. Luo, and C. Xu, "Learning a weakly-supervised video actor-action segmentation model with a wise selection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9901–9911.
- [14] L. Yu, P. Poirson, S. Yang, A. C. Berg, and T. L. Berg, "Modeling context in referring expressions," in *Proc. ECCV*, 2016, pp. 69–85.
- [15] R. Hu, M. Rohrbach, and T. Darrell, "Segmentation from natural language expressions," in *Proc. ECCV*, 2016, pp. 108–124.
- [16] C. Liu, Z. Lin, X. Shen, J. Yang, X. Lu, and A. Yuille, "Recurrent multimodal interaction for referring image segmentation," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 1280–1289.
- [17] L. A. Hendricks, O. Wang, E. Shechtman, J. Sivic, T. Darrell, and B. Russell, "Localizing moments in video with natural language," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 5803–5812.
- [18] J. Gao, C. Sun, Z. Yang, and R. Nevatia, "TALL: Temporal activity localization via language query," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 5267–5275.
- [19] K. Gavriluyk, A. Ghodrati, Z. Li, and C. G. M. Snoek, "Actor and action video segmentation from a sentence," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 5958–5966.
- [20] H. Wang, C. Deng, J. Yan, and D. Tao, "Asymmetric cross-guided attention network for actor and action video segmentation from natural language query," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 3939–3948.
- [21] H. Wang, C. Deng, F. Ma, and Y. Yang, "Context modulated dynamic networks for actor and action video segmentation with language queries," in *Proc. AAAI*, 2020, pp. 12152–12159.
- [22] B. McIntosh, K. Duarte, Y. S. Rawat, and M. Shah, "Visual-textual capsule routing for text-based video segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9939–9948.
- [23] D. He *et al.*, "StNet: Local and global spatial-temporal modeling for action recognition," in *Proc. AAAI*, Jul. 2019, vol. 33, no. 1, pp. 8401–8408.
- [24] X. Wang, R. B. Girshick, A. Gupta, and K. He, "Non-local neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2018, pp. 7794–7803.
- [25] W. Liu *et al.*, "SSD: Single shot MultiBox detector," in *Proc. ECCV*, 2016, pp. 21–37.
- [26] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 91–99.
- [27] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," in *Proc. CVPR*, 2017, pp. 2980–2988.
- [28] R. Hu, M. Rohrbach, J. Andreas, T. Darrell, and K. Saenko, "Modeling relationships in referential expressions with compositional modular networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1115–1124.
- [29] L. Yu *et al.*, "MAAttNet: Modular attention network for referring expression comprehension," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1307–1315.
- [30] P. Wang, Q. Wu, J. Cao, C. Shen, L. Gao, and A. V. D. Hengel, "Neighbourhood watch: Referring expression comprehension via language-guided graph attention networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 1960–1968.
- [31] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 3431–3440.
- [32] R. Yao, G. Lin, S. Xia, J. Zhao, and Y. Zhou, "Video object segmentation and tracking: A survey," *ACM Trans. Intell. Syst. Technol.*, vol. 11, no. 4, pp. 1–47, 2020.
- [33] P. O. Pinheiro, T.-Y. Lin, R. Collobert, and P. Dollár, "Learning to refine object segments," in *Proc. ECCV*, 2016, pp. 75–91.
- [34] J. Mao, J. Huang, A. Toshev, O. Camburu, A. Yuille, and K. Murphy, "Generation and comprehension of unambiguous object descriptions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 11–20.
- [35] X. Li and S. Jiang, "Bundled object context for referring expressions," *IEEE Trans. Multimedia*, vol. 20, no. 10, pp. 2749–2760, Oct. 2018.
- [36] S. Yang, G. Li, and Y. Yu, "Dynamic graph attention for referring expression comprehension," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 4644–4653.
- [37] E. A. Margfioyruay, J. C. Perez, E. Botero, and P. Arbelaez, "Dynamic multimodal instance segmentation guided by natural language queries," in *Proc. ECCV*, 2018, pp. 630–645.
- [38] R. Li *et al.*, "Referring image segmentation via recurrent refinement networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 5745–5753.
- [39] H. Shi, H. Li, F. Meng, and Q. Wu, "Key-word-aware network for referring expression image segmentation," in *Proc. ECCV*, 2018, pp. 38–54.
- [40] D.-J. Chen, S. Jia, Y.-C. Lo, H.-T. Chen, and T.-L. Liu, "See-through-text grouping for referring image segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 7454–7463.
- [41] L. Ye, M. Rochan, Z. Liu, X. Zhang, and Y. Wang, "Referring segmentation in images and videos with cross-modal self-attention network," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3719–3732, Jul. 2022.
- [42] Z. Hu, G. Feng, J. Sun, L. Zhang, and H. Lu, "Bi-directional relationship inferring network for referring image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 4424–4433.
- [43] S. Liu, T. Hui, S. Huang, Y. Wei, B. Li, and G. Li, "Cross-modal progressive comprehension for referring segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, May 13, 2021, doi: [10.1109/TPAMI.2021.3079993](https://doi.org/10.1109/TPAMI.2021.3079993).
- [44] J. Yang, Y. Huang, Z. Ma, and L. Wang, "CMF: Cascaded multi-model fusion for referring image segmentation," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2021, pp. 2289–2293.
- [45] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.

- [46] R. Socher, J. Bauer, C. D. Manning, and A. Y. Ng, "Parsing with compositional vector grammars," in *Proc. ACL*, 2013, pp. 455–465.
- [47] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein, "Learning to compose neural networks for question answering," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2016, pp. 1545–1554.
- [48] S. Kazemzadeh, V. Ordonez, M. Matten, and T. Berg, "ReferItGame: Referring to objects in photographs of natural scenes," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 787–798.
- [49] S. Yang, G. Li, and Y. Yu, "Graph-structured referring expression reasoning in the wild," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9952–9961.
- [50] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1532–1543.
- [51] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.
- [52] F. Faghri, D. J. Fleet, J. R. Kiros, and S. Fidler, "VSE++: Improving visual-semantic embeddings with hard negatives," in *Proc. BMVC*, 2018, pp. 1–14.
- [53] K. Niu, Y. Huang, W. Ouyang, and L. Wang, "Improving description-based person re-identification by multi-granularity image-text alignments," *IEEE Trans. Image Process.*, vol. 29, pp. 5542–5556, 2020.
- [54] H. Jhuang, J. Gall, S. Zuffi, C. Schmid, and M. J. Black, "Towards understanding action recognition," in *Proc. IEEE Int. Conf. Comput. Vis.*, Dec. 2013, pp. 3192–3199.
- [55] Y. Kong and Y. Fu, "Human action recognition and prediction: A survey," *Int. J. Comput. Vis.*, vol. 130, pp. 1366–1401, Mar. 2022.
- [56] Z. Li, R. Tao, E. Gavves, C. G. M. Snoek, and A. W. M. Smeulders, "Tracking by natural language specification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 6495–6503.
- [57] S. D. Jain, B. Xiong, and K. Grauman, "FusionSeg: Learning to combine motion and appearance for fully automatic segmentation of generic objects in videos," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2117–2126.
- [58] H. Xiao, J. Feng, G. Lin, Y. Liu, and M. Zhang, "MoNet: Deep motion exploitation for video object segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1140–1148.
- [59] A. Khoreva, R. Benenson, E. Ilg, T. Brox, and B. Schiele, "Lucid data dreaming for video object segmentation," *Int. J. Comput. Vis.*, vol. 127, no. 9, pp. 1175–1197, 2019.
- [60] A. Kirillov, Y. Wu, K. He, and R. Girshick, "PointRend: Image segmentation as rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9796–9805.



Kai Niu received the B.Eng. degree from the University of Science and Technology of China (USTC), Hefei, China, in 2015, and the Ph.D. degree from the Institute of Automation, Chinese Academy of Sciences, in 2020. He is currently an Associate Professor with the School of Computer Science, North-western Polytechnical University, Xi'an, China. His main research interests include multi-modal computing, computer vision, and machine learning.



Linjiang Huang received the B.Eng. and M.Eng. degrees from the Huazhong University of Science and Technology, China, in 2014 and 2017, respectively, and the Ph.D. degree from the Institute of Automation, Chinese Academy of Sciences, in 2020. He is currently a Postdoctoral Fellow with the Centre for Perceptual and Interactive Intelligence and Multimedia Laboratory, The Chinese University of Hong Kong. His main research interests include action recognition, action detection, machine learning, and computer vision.



Zhanyu Ma (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from the KTH Royal Institute of Technology, Sweden, in 2011. From 2012 to 2013, he was a Postdoctoral Research Fellow with the School of Electrical Engineering, KTH. From 2014 to 2019, he was an Associate Professor with the Beijing University of Posts and Telecommunications, Beijing, China. Since 2019, he has been a Professor with the Beijing University of Posts and Telecommunications. His research interests include pattern recognition and machine

learning fundamentals with a focus on applications in computer vision and multimedia signal processing.



Jianhua Yang received the B.Sc. degree from the Harbin Institute of Technology (HIT), Weihai, China, in 2015, and the M.Sc. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2018, where he is currently pursuing the Ph.D. degree. His main research interests include cross-modal learning, object localization, and segmentation.



Yan Huang received the B.Sc. degree from the University of Electronic Science and Technology of China (UESTC) in 2012 and the Ph.D. degree from the University of Chinese Academy of Sciences (UCAS) in 2017. He is currently an Associate Professor with the Institute of Automation, Chinese Academy of Sciences (CASIA). He has published more than 40 articles in the leading international journals and conferences, such as IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE (TPAMI), *International Journal of Computer Vision (IJCV)*, *NeurIPS*, *CVPR*, *ICCV*, and *ECCV*. His research interests include computer vision and cross-modal data analysis.



Liang Wang (Fellow, IEEE) received the B.Eng. and M.Eng. degrees from Anhui University, China, in 1997 and 2000, respectively, and the Ph.D. degree from the Institute of Automation, Chinese Academy of Sciences (CASIA), in 2004. From 2004 to 2010, he was a Research Assistant with Imperial College London, U.K., and Monash University, Australia; a Research Fellow with The University of Melbourne, Australia; and a Lecturer with the University of Bath, U.K. He is currently a Full Professor of the Hundred Talents Program with the National Laboratory of Pattern Recognition, CASIA. His major research interests include machine learning, pattern recognition, and computer vision. He is an IAPR Fellow.